

TGA C++ Kodstandard

Syfte

Syftet med en kodstandard är att förenkla ditt och andra programmerares arbete genom att ge dig en mall att utgå ifrån när du skriver kod.

Allmänt

1. Alla namn ska vara på korrekt stavad engelska.

Standarden

Det som följer Ska vara flyttat till de nya sidorna

Allmänt

Alla namn ska vara på korrekt stavad engelska.

Funktionsnamn

- Ska beskriva vad funktionen gör.
- Ska börja med stor bokstav.
- Formulera som regel med ett verb: Update, Print, Get, Create osv
- Första bokstaven i varje delord ska kapitaliseras.
- Ska vara ren text så långt som möjligt.

Bra exempel

```
void UpdateGameObject();  
void KillAvatar();  
void GetWreckedByChuckNorris();
```

Dåliga exempel

```
void Spelarestudsar();  
void L337HaxxÄgerdig();  
void NAMNInput();
```

Medlemsvariabler i klasser

- Namnet ska börja med `my`.
- Alla delord efter det första ska börja med stor bokstav.
- Ska beskriva vad den används till.
- Ska vara privata och läsas / skrivs med Getters och Setters.

Bra Exempel

```
int myNumberOfLives;  
bool myIsAlive;  
Sprite mySprite;
```

Dåliga Exempel

```
int RandomSeed;  
float datum;  
Sprite bild;
```

Getters i Klasser och Structs

- Om Gettern returnerar en medlemsvariabel av en standardtyper (bool, int, float) bör den returneras som värde
- Om Gettern returnerar en medlemsvariabel av annan typ bör den returneras som const reference
- Om Gettern returnerar ett beräknat värde skall det returneras by value

Exempel

```
const std::vector<GameObject>& GetGameObjects() const;  
int GetCalculatedDamage() const;
```

Lokala Variabler

- Ska börja med liten bokstav.
- Ska beskriva vad variabeln används till.
- Innehåller normalt ett beskrivande substantiv: vector, enemy, character, counter, osv.
- Alla delord efter det första ska börja med stor bokstav.
- Gäller även om variablerna är konstanta

Bra exempel

```
Avatar playerAvatar;  
Weapon fistOfChuckNorris;  
Container moneyBag;  
int index;  
const int speed;
```

Dåliga exempel

```
Avatar a;  
int i;  
char *DUMMY;  
float Test;  
Object Foo;
```

Funktionsargument

- Ska inledas med `a` som i 'argument'.
 - `an` eller `some` är också godkänt för grammatisk korrekthet.
- I övrigt samma regler som för andra variabelnamn.

Bra exempel

```
aPlayerAvatar  
aFistOfChuckNorris  
someMoney  
anOperator  
someFish
```

Dåliga exempel

AaaDUMMY

SomeFistMoneyKlasser

- Funktionsargument som inte är standardtyper (bool, int, float) bör skickas som const reference om möjligt **Exempel** `bool CreatePlayer(const std::string& aPlayerName, float aPlayerHealth);`
- Funktionsargument som fungerar som extra utdata skall ha prefixet `out` om den kommer att initieras och returneras av funktionen.
- Funktionsargument som både kommer att läsas och skrivas som extra utdata skall ha prefixet `inout`. **Exempel** `bool CreatePlayer(const std::string& aPlayerName, float aPlayerHealth, std::shared_ptr<Player>& outPlayer);` `size_t UpdateGameObjects(std::vector<GameObject>& inoutGameObjects);`

Klasser

- Klassnamn ska börja med stor bokstav.
- Klassnamn ska beskriva klassens funktion.
- Klassnamn ska ej vara plural.
- Innehåller normalt ett beskrivande substantiv: Vector, Enemy, Character, Counter, osv.
- Namn på medlemsvariabler ska börja med `my`
- Namn på statiska medlemsvariabler ska börja med `our`
- Funktionsnamn följer samma regler som alla funktioner.

Bra exempel

```
class Vehicle
{
    public:
        Vehicle(Model aVehicleModel);
        ~Vehicle();

    private:
        int mySpeed;
        const Model myModel;
        static const int ourMaxSpeed = 15;
};
```

Structs

Namn på struct

- Ska börja med stor bokstav.
- Första bokstaven i varje delord ska kapitaliseras.
- Används fördelaktigt som data container eller POD (Plain Old Data) objekt.

Variabler i en struct

- Ska börja med liten bokstav. I övrigt som övriga variabler.
- Namn på privata medlemsvariabler ska börja med my. Men bör undvikas. Använd some regel klass istället
- Funktionsnamn följer samma regler som alla funktioner

Måsvingar på ny rad

Bra exempel

```
struct Car
{
    string brand;
    string model;
    int releaseYear;

    private:
        bool myIsDestroyed;
};
```

Övriga variabler

- Globala variabler som ej är konstanta
 - Ska ha `global` som prefix.
 - Om de är begränsade till en fil ska de ha `loc` som prefix (dvs de är globala variabler deklarerade som static).
 - I övrigt som övriga variabler.
- Observera att typen inte behöver återspeglas i namnet som i t.ex. hungarian notation.
 - Det behövs inte i och med att ett modernt IDE kan göra lookups direkt när man för muspekaren över variabeln. Koden blir mer läsbar utan mystiska bokstavskombinationer i början av variabelnamnet.
- if-satser ska alltid ha `{` och `}` om inte speciella skäl existerar.
- Variabler bör allokeras så sent som möjligt av prestandaskäl.
- Måsvingar
 - Skiljer sig från boken C++ primer.
 - En måsvinge ska inte vara på samma rad i koden som skapar den då det blir svårare att följa dem i koden.

Bra exempel

```
if(a > b)
{
    // Code
}

while(a > b)
{
    // Code
}
```

Dåliga exempel

```
if( a > b ) {
    // Code
}

if( a > b )
    // Single statement;

if( a > b ) // Single statement;

while( a > b ) {
    // Code
}
```

Casts

C-style casts tillåter vilka casts som helst utan varningar. I och med detta så är de en vanlig orsak till problem och skall undvikas så långt som möjligt. C++ har `static_cast`, `reinterpret_cast`, `dynamic_cast` och `const_cast` som ersätter c-style casts men ger bättre felhantering.

- Försök undvika att göra casts.
- Använd aldrig c-style casts.
- Om en cast måste göras ska C++ style casts, vanligtvis `static_cast`, användas.

Bra exempel

```
float floatNumber = 10.2f;
int integerNumber = static_cast<int>(floatNumber);
```

Dåligt exempel

```
float floatNumber = 10.2f;
int integerNumber = (int)floatNumber;
```

#include

Det finns inga regler i skolans kodstandard beträffande `#include`. Däremot finns det några saker att tänka på. Citattecken ("") eller taggar (<>)? Skillnaden mellan dessa handlar om var preprocessor söker efter filen som ska inkluderas. När filen väl inkluderas är det ingen skillnad.

- Citattecken
 - Om man använder citattecken så kommer preprocessor i börja med att leta efter filen i den katalog som filen med `#include`-direktivet påträffades. Därefter kommer den att leta i de kataloger som specificerats i kompilatorn.
- Vinkelklamrar
 - Om man använder dessa så kommer preprocessor först att leta efter filen i de kataloger som kompilatorn känner till.

Använd citattecken för alla includes inom projektet och vinkelklamrar för övriga includes.

Konstanter

- Använd när möjligt `const` eller `constexpr` istället för `defines`
- Namnges med stora bokstäver med _ mellan ord
- Ska beskriva vad konstanten används till.

Exempel `static inline constexpr int MAX_PLAYERS = 8;` `const int MAX_PLAYERS = 8;`

- Defines namnges på samma sätt: **Bra exempel**

```
#define MAX_PLAYERS = 8
#define PI 3.14159265359
```

Dåligt exempel

```
#define Monkeys 10
#define starting_money 42
```

- Samma namngivning gäller för konstanter i klasser (som då är `static const` eller `static constexpr`)

Defines av Funktioner

- Bör undvikas om möjligt. Skriv alltid namnet med versaler.
- Dokumentera makrofunktionen.

Exempel

```
// Selects the max value of A or B.  
#define MAX(A, B) A > B ? A : B
```

C-style enums (inte enum class)

Namn på enum

- Ska börja med stor bokstav.
- Första bokstaven i varje delord ska kapitaliseras.
- Ska ej vara plural.

Konstanterna i en enum

- Ska börja med stor bokstav.
- Första bokstaven i varje delord ska kapitaliseras.
- Ska prefixas med namnet på enum + ett underscore.

Måsvingar på ny rad. Ha gärna en speciell count-konstant sist.

Bra exempel

```
enum ColorChannel  
{  
    ColorChannel_Red,  
    ColorChannel_Green,  
    ColorChannel_Blue,  
    ColorChannel_Alpha,  
    ColorChannel_Count  
};
```

Enums (enum class, C++11 enum)

Namn på enum

- Ska börja med stor bokstav.
- Första bokstaven i varje delord ska kapitaliseras.
- Ska ej vara plural.

Konstanterna i en enum

- Ska börja med stor bokstav.
- Första bokstaven i varje delord ska kapitaliseras.

Måsvingar på ny rad. Ha gärna en speciell count-konstant sist.

Bra exempel

```
enum class ColorChannel
{
    Red,
    Green,
    Blue,
    Alpha,
    Count
};
```

Scopes (if, switch...case)

- Både if och case skall ha scope definierat
- Gör koden lättare att läsa
- Undviker underliga problem och optimeringsbuggar i switch...case

Exempel

```
if(myLives <= 0)
{
    SetGameState(GameState::GameOver);
}
```

```
switch(myGameSate)
{
    case MenuScreen:
    {
        DrawMenuScreen();
        break;
    }
}
```

```
}  
case GameOver:  
{  
    DrawGameOverScreen();  
    break;  
}  
}
```

Revision #6

Created 24 October 2024 11:00:45 by Daniel Borgshammar

Updated 21 August 2025 15:09:27 by Albin Esko